# Faculty of Technology and Engineering

Date: 21/12/2025

| | | | | | |
|---|---|---|---|---|---|
| Academic Year | : | 2025-2026 | Semester | : | 2 |
| Course code | : | CEUC102 | Course name | : | Programming with C++ |

## Practical List

| Sr. No. | Practical Title | No. of Sub practical | Page No. | CO | Student Engagement |
|---|---|---|---|---|---|
| 1 | Principles of object-oriented Programming | 2 | 3 | CO1 | 1.1 - 2 hr<br>1.2 - 1 hr<br>1 hr – Fundamental of Competitive Programming |
| 2 | Class & Object | 5 | 6 | CO2 | 2.1 – 0.5 hr<br>2.2 – 1 hr<br>2.3 – 1 hr<br>2.4 – 1.5 hr<br>2.5 – 1 hr<br>Evaluation – 1 hr |
| 3 | Functions, Templates & Strings | 5 | 12 | CO3 | 3.1 – 1 hr<br>3.2 – 1 hr<br>3.3 – 0.5 hr<br>3.4 – 1 hr<br>3.5 – 1.5 hr<br>Evaluation – 1 hr |
| 4 | Pointers and DAM | 3 | 18 | CO3 | 4.1 – 1 hr<br>4.2 – 1 hr<br>4.3 – 1 hr<br>Evaluation – 1 hr |
| 5 | Stream Input/Output and File Processing | 4 | 22 | CO3 | 5.1 – 1 hr<br>5.2 – 0.75 hr<br>5.3 – 2 hr<br>5.4 – 1 hr<br>Evaluation – 1.25 hr |
| 6 | Inheritance | 4 | 26 | CO4 | 4.1 – 1.5 hr<br>4.2 – 1.5 hr<br>4.3 – 1.5 hr<br>4.4 – 1.5 hr<br>Evaluation – 2 hr |

| | | | | | |
|---|---|---|---|---|---|
| 7 | Polymorphism | 6 | 32 | CO4 | 7.1 – 1 hr<br>7.2 – 1 hr<br>7.3 – 1 hr<br>7.4 – 1 hr<br>7.5 – 1 hr<br>7.6 – 1 hr<br>Evaluation – 2 hr |
| 8 | Exception Handling | 4 | 39 | CO5 | 8.1 – 0.5 hr<br>8.2 – 1 hr<br>8.3 – 1 hr<br>8.4 – 2 hr<br>Evaluation – 1.5 hr |
| 9 | Standard Template Library | 4 | 44 | CO6 | 9.1 – 1 hr<br>9.2 – 0.5 hr<br>9.3 – 0.5 hr<br>9.4 – 1 hr<br>Evaluation – 1 hr |

**Tools/Technology To Be Used:** C++ Editor, VS Code, jDoodle

**Rubrics for Practical Evaluation:**

| Sr. No. | Criterion | Marks | What to check |
|---|---|---|---|
| 1 | Correctness of Implementation | 3 | Program meets the problem statement: required features implemented, algorithms produce correct results for provided test cases. No logical errors. |
| 2 | Design & Use of Concepts | 2 | Appropriate use of the concepts (e.g., OOP principles, pointers/dynamic memory, STL containers, exceptions, file I/O, templates). Class/function decomposition and modularity. |
| 3 | Validation, Robustness & Output Formatting | 2 | Proper input validation and error handling (i.e. boundary cases, invalid input, file-not-found, exception handling). Output is neat and readable (tables, labels, spacing). Naming conventions and comments are appropriately followed. |
| 4 | Viva / Conceptual Understanding | 3 | Student can explain key design choices, how main functions work (Short viva or Q&A) |
| | **Total** | 10 | |

**Practical CIE:** 23/02/2026 to 27/02/2026, Syllabus: Practical-1,2,3

**Internal CIE:** 13/04/2026 to 17/04/2026, Syllabus: Practical-4,5,6,7

# 1. Principles of object-oriented Programming

## Practical 1.1

**Problem Definition**

A digital payment company wants to develop a simple account management module for its users. Each user maintains a digital wallet that allows loading money, transferring money to another user, and checking the current wallet balance.
Each wallet has:

- A unique wallet ID
- User name
- Current balance

The system should allow:

- Loading money into the wallet
- Transferring money to another wallet only if sufficient balance exists
- Displaying wallet details on request

If transfer amount exceeds the current balance, the system should show an appropriate error message. The solution must be implemented using **Object-Oriented Programming principles**, especially encapsulation and data hiding.

**Key Questions:**

1. How is **encapsulation** used to protect user balance?
2. How are **member functions** used to validate transactions?
3. How does the system ensure that balance cannot be modified directly?

**Supplementary Problems:**

- Extend the system to maintain a **transaction history** (last 5 transactions).
- Add a feature to apply a **cashback percentage** for every wallet load.
- Introduce a feature to **freeze/unfreeze** user wallets.

**Key Skills to be addressed**

- Class design and abstraction
- Encapsulation and data protection

- Object interactions and method invocation

**Applications:**

- E-wallet system
- FinTech applications
- Digital KYC-based account management

**Learning Outcome**

Students will be able to implement OOP concepts through wallet management, apply data protection techniques, and design methods for transactional checks.

**Post Laboratory Work Description**

Students must write a reflection summarizing:

- How encapsulation protects user data
- Why data should not be modified directly
- What test cases they used to verify the transfer function

## Practical 1.2

**Problem Definition**

A small library wants to digitize its system to manage its collection of books. Every book has:

- Book ID
- Title
- Author
- Number of available copies

The system must allow:

- Adding new books to the library
- Issuing a book to a student (only if copies are available)
- Returning issued books
- Displaying total books in the library

The library committee wants students to implement the system using both **procedural programming** and **object-oriented programming**, and then compare both approaches in terms of code reusability and complexity.

**Key Questions / Analysis / Interpretation to be evaluated**

1. What limitations are visible in the procedural version?
2. How does OOP improve scalability and code maintainability?

3. How do constructors and member functions simplify book operations?

**Supplementary Problems**

- Add search functionality (search by author or title).
- Add due date and fine calculation for late returns.
- Create a user class for students who borrow books.

**Key Skills to be addressed**

- Class design and modularity
- Comparison of procedural vs OOP thinking
- Handling real-world constraints in programming
- Implementation of basic CRUD operations

**Applications**

- Library management systems
- Bookshop inventory
- Digital resource tracking portals

**Learning Outcome**

Students will understand how procedural and object-oriented approaches differ in real applications and will be able to design an OOP-based book management system.

**Post Laboratory Work Description**

Students must submit:

- A comparison table between procedural vs OOP approach
- Test cases for issuing and returning books
- Reflection on where OOP gives higher flexibility

# 2. Class & Object

## Practical 2.1 – Rectangle Management System

### Problem Definition

A local construction company frequently deals with rectangular plots and structures of varying dimensions. To streamline their planning and estimation processes, the company requires a simple system to manage and analyze rectangular shapes efficiently.
The system must handle multiple rectangles, each with distinct dimensions. For each rectangle, the length and width need to be defined and stored securely. Additionally, the system should calculate:

- **Area**, to estimate usable space or material coverage
- **Perimeter**, to estimate boundary lengths or material requirements

The system should allow updating dimensions anytime and should manage multiple rectangle records, performing calculations for each and displaying the results clearly.

### Key Questions / Analysis / Interpretation

1. How are object attributes (length, width) encapsulated to prevent invalid values?
2. What method(s) compute area and perimeter, and why should they be separate functions?
3. How can an array of objects simplify handling multiple rectangles?

### Supplementary Problems

- Add a function to check whether a rectangle is a **square**.
- Add comparison between two rectangles (larger area).
- Add input validation: no negative dimensions allowed.

### Key Skills Addressed

- Class creation and object instantiation
- Access specifiers, data encapsulation
- Modelling real-world problems

### Applications

- Construction planning
- Architecture design software
- Land measurement tools

**Learning Outcome**

Students can design classes, create objects, access attributes using member functions, and compute geometric properties using OOP.

**Post Laboratory Work**

Students must submit rectangle test cases demonstrating:

- large dimensions
- equal sides
- invalid input handling

## Practical 2.2 – Student Academic Record System

**Problem Definition**

A university requires a system to digitize student academic records. Each student's roll number, name and marks in three subjects must be stored. The system must calculate and display each student's average marks.
The system should allow two types of initialization:

- **Default values** for testing
- **User-provided input values**

Multiple student records should be created and displayed to test accuracy of stored data and computed averages.

**Key Questions / Analysis / Interpretation**

1. Why is constructor overloading suitable for this system?
2. How to prevent invalid marks using encapsulation?
3. What patterns emerge when storing multiple objects?

**Supplementary Problems**

- Determine the student with highest average.
- Assign grade based on average marks.
- Validate marks (0–100 only).

**Key Skills Addressed**

- Constructors and object initialization
- Multidata object management using arrays
- Basic computations inside a class

**Applications**

- Academic MIS systems
- Result generation software

**Learning Outcome**

Students understand constructor overloading and object arrays while designing academic data systems.

**Post Laboratory Work**

Submit a comparison table of default vs parameterized constructor use-cases.

## Practical 2.3 – Digital Savings Account System

**Problem Definition**

A growing FinTech startup wants to create a **Digital Savings Account System** for its mobile banking app. Each digital account will store details such as:

- Customer name
- Unique account ID
- Current balance

The system must allow:

- Creating accounts with **zero balance** or **custom initial balance**
- Depositing money
- Withdrawing money with validation (no overdraft)
- Displaying full account summary for users

To prepare for real deployment, the development team wants to simulate:

- Multiple account creations
- Sequences of deposits and withdrawals
- Edge cases such as withdrawing more than the available balance
- Accurate summary generation for each account

The goal is to build a modular class-based solution with strong input validation, ensuring reliability and correctness.

**Key Questions / Analysis / Interpretation**

1. Why should balance be private?
2. Which operations require validation and why?
3. How can the system be scaled for multiple users?

**Supplementary Problems**

- Add a feature to apply **interest** to the balance.
- Add a function to classify accounts as **active/inactive**.
- Implement a mini transaction log (last 3 transactions).

**Key Skills Addressed**

- Object-oriented modelling
- Access specifiers
- Method design with validation

**Applications**

- Banking apps
- Wallet services
- FinTech payment systems

**Learning Outcome**

Students will implement deposit/withdraw systems with correctness checks, design co **Post**

**Post Laboratory Work**

Students must simulate:

- At least 3 accounts
- 2 deposits and 2 withdrawals per account
- One invalid withdrawal case

## Practical 2.4 – Inventory Management System

**Problem Definition**

A retail store requires a digital system to manage its inventory of items. Each item has:

- Unique item ID
- Item name
- Price
- Quantity in stock

The system must allow:

- Initialization using default or parameterized constructors
- Increasing stock when new shipments arrive
- Decreasing stock when items are sold
- Validating stock before sale
- Displaying item details for decision-making

The system must also support managing multiple items at once.

**Key Questions / Analysis / Interpretation**

1. How does encapsulation secure item details?
2. What checks are required before selling an item?
3. How can an array of objects represent a product catalog?

**Supplementary Problems**

- Display items with stock < 5 (low inventory alert).
- Update prices during discount sale.
- Sort items based on quantity.

**Competitive Programming Task**

**Given N items, find the item with the maximum stock quantity**.

**Key Skills to be Addressed**

- Class creation
- Validation and error handling
- Using arrays of objects for comparison tasks

**Applications**

- Retail inventory systems
- Warehouse stock management
- E-commerce product catalogues

**Learning Outcome**

Students will implement a simple inventory system and solve a basic competitive-programming-style logic problem using objects.

**Post Laboratory Work Description**

Students must test selling, restocking, and run the mini-task for at least 5 items.

## Practical 2.5 – Loan EMI Management System

**Problem Definition**

A regional bank wants a system to manage loan accounts. Each loan must store:

- Loan ID
- Applicant name
- Total loan amount
- Annual interest rate
- Loan tenure (months)

The system must calculate the **Monthly EMI** using the formula:

$$EMI = \frac{P \times R \times (1+R)^T}{(1+R)^T - 1}$$

The system should initialize data using both default and parameterized constructors and display complete loan details.

**Key Questions / Analysis / Interpretation**

1. How does using a class simplify EMI computation?
2. Why must EMI calculation be implemented as a member function?
3. What checks must be applied to loan amount, interest rate, and tenure?

**Supplementary Problems**

- Calculate total repayment amount.
- Recalculate EMI after interest rate revision.
- Validate that loan amount and tenure are positive.

**Competitive Programming Task**

**Given N loan accounts, determine the account with the 2nd highest EMI.**
This tests comparison logic and object handling.

**Key Skills to be Addressed**

- Implementing formulas in OOP
- Object comparisons
- Input validation

**Applications**

- Loan processing software
- EMI calculators
- Financial planning systems

**Learning Outcome**

Students learn to compute EMI and perform comparisons using object attributes.

**Post Laboratory Work Description**

Students must compute EMI for at least 4 loans and determine which loan has the highest EMI.

**3. Function and Recursion**

## Practical 3.1 – Payroll Salary Calculation Using Inline Function

**Problem Definition**

A growing organization wants to automate its payroll system to efficiently compute employee salaries. Each employee's record must store the **name**, **basic salary**, and **bonus amount**.
While most employees receive a default bonus, exceptional performers may receive a customized bonus.
To enable instant computation of the salary, an **inline function** will calculate **total salary = basic salary + bonus**.
Employee details must be initialized through a constructor, using either default or customized bonus amounts.
As the organization expands, dynamic storage of employee records is required to support scalability.
The system must display complete employee details along with their total salary.

**Key Questions / Analysis / Interpretation**

1. How does an inline function improve performance for frequently used operations?
2. Why is dynamic storage suitable for handling a variable number of employee records?
3. How should constructors be designed to support both default and custom bonus initialization?

**Supplementary Problems**

- Add validation ensuring salary cannot be negative.
- Introduce grade category based on salary.
- Add a function to update bonus during appraisal.

**Key Skills to be Addressed**

- Use of inline functions
- Function design inside a class
- Dynamic allocation techniques

**Applications**

- Payroll systems

- HR analytics
- Salary slip generation tools

**Learning Outcome**

Students will implement inline functions, constructors, and dynamic memory to manage multiple employee records.

**Post Laboratory Work Description**

Students must generate payroll for at least 5 employees, using a mix of default and custom bonuses.

## Practical 3.2 – Recursive vs Iterative Array Sum

**Problem Definition**

A software development company wants to compare the performance of recursive and iterative approaches for computing the **sum of integers in an array**.
The system should dynamically accept the array size and allow users to input all elements.
Two functions must be implemented:

- A **recursive** function to compute the sum by dividing the array into smaller subproblems
- An **iterative** function to compute the same sum for comparison

The objective is to evaluate execution time, memory usage, and complexity between recursion and iteration.

**Key Questions / Analysis / Interpretation**

1. What is the base case and recursive case in summing elements of an array?
2. How does recursion affect memory due to stack frames?
3. When should recursion be preferred over iteration?

**Supplementary Problems**

- Find maximum element using recursion and iteration.
- Modify recursion to compute prefix sums.
- Analyze stack depth for large input sizes.

**Key Skills to be Addressed**

- Function design using recursion
- Iterative vs recursive complexity analysis
- Dynamic array handling

**Applications**

- Algorithm analysis
- Data processing systems
- Coding interview preparation

**Learning Outcome**

Students will understand recursive decomposition and compare efficiency with iterative implementation.

**Post Laboratory Work Description**

Students must compare execution time for both methods and write a short reflection.

## Practical 3.3 – Updated Problem Definition: Enhanced Account Tracking System

**Problem Definition**

A community bank wants to upgrade its digital account management system to improve efficiency and support reliable customer service. Each customer account must store:

- Account number
- Account holder's name
- Current balance

The system should allow customers to **transfer money securely** between accounts with full validation to prevent invalid transactions.
The bank also wants the ability to track the **total number of accounts created** using a **static data member**, which is essential for reporting and growth analysis.
Accounts must be stored dynamically to support scalability as the bank grows. The system should allow adding, updating, and retrieving account information efficiently, with complete summaries displayed for each account.

**Key Questions / Analysis / Interpretation**

1. How does a static data member help track total accounts?
2. Why is validation necessary during fund transfer operations?
3. How can dynamic allocation improve scalability?

**Supplementary Problems**

- Add transaction count per account.
- Add a function to apply service charges.
- Introduce minimum-balance validation logic.

**Key Skills to be Addressed**

- Static data members and functions
- Object interaction for money transfer
- Dynamic storage of objects

**Applications**

- Banking account systems
- Digital wallet management
- Customer transaction platforms

**Learning Outcome**

Students will understand static members, inter-object communication, and dynamic storage in financial applications.

**Post Laboratory Work Description**

Students must create at least 4 accounts dynamically, record a fund transfer sequence, and display final account summaries.

## Practical 3.4 – Function Templates

**Problem Definition**

A technology firm wants to develop a reusable, type-independent solution for operating on collections of different data types such as **integers, floating-point numbers, and characters**.
To avoid code duplication, the solution must use **function templates** that allow the same logic to run on multiple data types.
The system must perform the following tasks on template-based arrays stored in **simple static arrays**:

- Find the **maximum value**
- Reverse the array
- Display the elements

**Competitive Programming Task**

**Find Leader Elements in an Array (Template-Based)**

A leader element in an array is one that is **greater than all elements to its right**.
Students must write a **template function** to print leader elements for:

- Integer array
- Float array
- Character array (where comparison is based on ASCII values)

*For Example:*
Input: arr = [16, 17, 4, 3, 5, 2]
Output: 17 5 2

**Key Questions / Analysis / Interpretation**

1. How do template functions eliminate redundancy for operations across multiple data types?
2. Why are static arrays suitable at this stage instead of advanced containers?
3. How is the leader-element condition checked efficiently?
4. What are possible edge cases when comparing characters using templates?

**Supplementary Problems**

- Use a template function to find the **minimum element**.
- Use a template function to **search an element** in an array.
- Use a template function to **count occurrences** of a given value.

**Key Skills to be Addressed**

- Function template syntax and instantiation
- Array traversal logic
- Enhanced problem-solving (leader logic)
- Type-generic programming

**Applications**

- Generic utility libraries
- Competitive programming basics
- Reusable algorithm modules

**Learning Outcome**

Students will learn to write template-based functions that operate on different data types and apply competitive-programming logic such as leader-element detection.

**Post Laboratory Work Description**

Students must test all template functions using arrays of **int, float, and char**, and write outputs from:

- Maximum value function
- Reverse array
- Leader elements

## Practical 3.5 – Super Digit Computation Using Recursion

**Problem Definition**

A data analytics company was tasked with developing a unique digital signature system based on the concept of "super digits." The system required finding a single-digit representation of a given number through recursive digit summation. The algorithm was defined as follows:

If the number has only one digit, it is its super digit. Otherwise, the super digit is the super digit of the sum of its digits, repeated recursively until a single digit is obtained.

The challenge involved an additional complexity—constructing the number by concatenating a given string representation of an integer multiple times. For example, if the number n was represented as a string and concatenated k times, the super digit of the resulting number needed to be calculated.

For instance:

Given n = 9875 and k = 4, the number is represented as 9875987598759875.

The sum of digits in this number is calculated recursively until a single digit remains:

9 + 8 + 7 + 5 + 9 + 8 + 7 + 5 + 9 + 8 + 7 + 5 + 9 + 8 + 7 + 5 = 116

1 + 1 + 6 = 8

The super digit is 8.

The system was required to handle large numbers efficiently by leveraging mathematical insights rather than explicitly constructing large concatenated strings. This case study called for implementing a recursive solution to calculate the super digit, supported by a mathematical approach to optimize the handling of repeated sums.

**Key Questions / Analysis / Interpretation**

1. What is the base case for recursive super digit calculation?
2. Why is explicit string concatenation inefficient?
3. How can mathematical reduction optimize super digit computation?

**Supplementary Problems**

- Count recursion depth for different inputs.
- Track intermediate digit sums.
- Modify recursion to stop after a fixed number of levels.

**Key Skills to be Addressed**

- Recursive problem solving
- Mathematical reduction techniques
- String and digit manipulation

**Applications**

- Digital signatures
- Hashing techniques
- Data fingerprinting systems

**Learning Outcome**

Students will implement recursive logic for super digits and apply mathematical optimization for large numeric inputs.

**Post Laboratory Work Description**

Students must compute super digits for at least 5 test cases, including large n and large k.

# Practical 4: Pointers & DAM

## Practical 4.1: Dynamic Array Management Using Pointers

**Problem Definition**

A software development team needs a lightweight, custom-built solution to manage a growing list of numerical records without relying on STL containers like vector. The system must simulate the behavior of a resizable array using **raw pointers** and **dynamic memory allocation (new, delete)**.

The array should support three core operations:

1. **Insert an element at the end**
2. **Delete an element from a specified position**
3. **Display the current array contents**

**Key Questions / Analysis / Interpretation**

1. How does dynamic memory allocation (new, delete) allow an array to grow and shrink?
2. Why must elements be manually copied when resizing a non-STL array?
3. What edge cases arise during deletion (invalid positions, empty array, etc.)?

**Supplementary Problems**

- Add a function to **insert an element at a specific index**.
- Add a function to **search** for an element and return its position.
- Implement a function to **count occurrences** of a given value.
- Add **boundary checks** for invalid delete/insert positions.

**Key Skills to be Addressed**

- Pointer manipulation
- Dynamic memory management (new, delete, nullptr)
- Manual array resizing
- Basic time complexity understanding for insert/delete operations

**Applications**

- Simulation of dynamic arrays used internally in data structures
- Understanding how STL vector works behind the scenes

- Useful in embedded systems where STL may be restricted

**Learning Outcome**

Students will be able to implement a dynamic, resizable array manually using pointers, demonstrating understanding of memory allocation, reallocation, and element manipulation in C++.

**Post Laboratory Work Description**

Students must test:

- At least 5 insertions
- At least 3 deletions at various positions
- Printing the array after each operation
- Reflection on how STL vector automates this logic

## Practical 4.2: Point Class with Chainable Movement Using this Pointer and Pass-by-Reference

**Problem Definition**

A graphical software team needs a basic **Point** class to represent 2D coordinates for various geometric operations. Each point must store two attributes:

- **x-coordinate**
- **y-coordinate**

The class must include a function:

move(int dx, int dy)

which shifts the point by offsets dx and dy.
A key requirement is that the **move() function must be chainable**, allowing syntax such as:

p.move(2, 3).move(-1, 4);

This behavior should be implemented using the **this pointer** to return the current object reference.

Additionally, the practical must demonstrate the concept of **pass-by-reference using pointers**, showing how an object can be updated when passed to a function via pointer parameters.

**Key Questions / Analysis / Interpretation**

1. How does the this pointer allow member function chaining in C++?
2. Why does passing an object using pointers allow direct modification of its attributes?
3. How do coordinate updates reflect changes in the point's position?

**Supplementary Problems**

- Add a function to calculate the **distance from origin**.
- Add a function to shift the point to **absolute coordinates**.
- Create a function using pointer parameters to **reset** the point to (0,0).
- Add validation to detect overflow if coordinates exceed certain limits.

**Key Skills to be Addressed**

- Understanding and usage of the this pointer
- Method chaining (fluent interface design)
- Passing objects by pointer (pass-by-reference)
- Updating object states correctly using pointer dereferencing

**Applications**

- Game development (movement of objects)
- Graphics programming (points, shapes, transformations)
- Coordinate geometry calculators
- Simulation and modeling software

**Learning Outcome**

Students will learn how the this pointer enables method chaining, how to update objects through pointers, and how to design interactive behaviors within classes.

**Post Laboratory Work Description**

Students must:

- Create a point object
- Apply at least **three chained move() operations**
- Update the point using a **separate function that receives the object by pointer**
- Display the point's final coordinates

## Practical 4.3: Merge Two Sorted Arrays Using Dynamic Memory Allocation

**Problem Definition**

A data processing system requires merging two sorted datasets into a single sorted output. Since built-in containers and STL vectors are not allowed, the merging must be implemented manually using **dynamic memory allocation (new and delete)**.

The program will:

1. Read two integers **n** and **m**, representing the sizes of the two sorted arrays.
2. Dynamically allocate arrays of size **n** and **m** using new.
3. Merge both arrays into a **new dynamically allocated sorted array** of size **n + m**.
4. Display the merged sorted array.

This practical helps students understand how merging works at a low level without STL, and how dynamic memory can be used to manage flexible array sizes.

**Key Questions / Analysis / Interpretation**

1. How does merging two sorted arrays work using linear traversal?
2. Why do we need dynamic memory (new / delete) when array sizes are entered at runtime?
3. What happens to memory when we delete dynamically allocated arrays?

**Supplementary Problems**

- Modify the program to remove duplicate elements after merging.
- Extend the logic to merge **three** sorted arrays.
- Count how many comparisons and moves were made during the merge.
- Implement descending order merge.

**Key Skills to be Addressed**

- Dynamic memory management
- Manual array merging (without STL)
- Two-pointer technique for merging
- Handling runtime-determined array sizes

**Applications**

- Merge step in **Merge Sort**
- Combining ordered data streams
- Database query result merging
- Data preprocessing in analytics

**Learning Outcome**

Students will learn to dynamically allocate arrays, merge sorted lists using efficient logic, and manage memory manually using new and delete.

**Post Laboratory Work Description**

Students must test the merge program with at least 3 different test cases, including:

- Arrays of unequal sizes
- Arrays with duplicate values
- Edge cases (n=1, m=1), (n=0 not allowed here but can be discussed)

# Practical 5: Stream Input/Output and File Processing

## Practical 5.1: Word Frequency Counter

### Problem Definition

The task is to write a C++ program that reads a complete paragraph from user input using **unformatted input** (getline), stores all words manually in a **basic array of strings**, and counts the frequency of each word **without using any STL containers**.

The program must count words **case-insensitively** so that words like "Hello", "hello", and "HELLO" are treated as the same word.
The implementation must rely entirely on manually managed arrays, loops, and comparisons.

### Key Questions / Analysis / Interpretation

1. How can an entire paragraph be read using unformatted input (getline)?
2. How do we split the paragraph into individual words?
3. How does manual case conversion (tolower) help in case-insensitive comparison?
4. How can word frequencies be counted using simple arrays and nested loops?

### Supplementary Problems

- Remove punctuation before counting words (optional challenge).
- Identify the **most frequent** word manually.
- Count the **total number of unique words**.
- Print the words in the order of their first appearance.

### Key Skills to be Addressed

- Manual string parsing and word extraction
- Case conversion and string comparison
- Frequency counting using arrays
- Basic text preprocessing
- Memory and array size management without STL

### Applications

- Basic text analytics
- Search indexing
- Keyword extraction
- Natural language preprocessing fundamentals
- File content analysis

**Learning Outcome**

Students will learn how to perform frequency counting with arrays and implement case-insensitive comparison.

**Post Laboratory Work Description**

Students must:

- Test the program with at least 3 different paragraphs
- Show printed output of:
  - Extracted words
  - Word frequencies
- Write a 40–60 word reflection on challenges faced without STL

## Practical 5.2: File Reading and Counting Characters, Words, and Lines

**Problem Definition**

Write a C++ program to read a text file containing multiple lines and manually count the **total number of characters, words, and lines**. The program must process the file line-by-line using basic string operations and handle errors if the file does not exist or fails to open.

This approach will involve reading each line using unformatted input, scanning characters manually to count words and characters, and maintaining counters using simple variables and arrays.

**Key Questions / Analysis / Interpretation**

1. How does ifstream read files line by line without STL containers?
2. What logic is needed to detect word boundaries manually?
3. How do we detect file-open failures and handle them gracefully?
4. What is the difference between counting characters with and without whitespace?

**Supplementary Problems**

- Count the number of blank lines in the file.
- Count the frequency of vowels and consonants manually.
- Detect the longest line in the file.
- Reverse each line using manual character swapping.

**Key Skills to be Addressed**

Manual string parsing, file handling, boundary detection for words, character scanning, and working without STL dependency.

**Applications**

Text file analysis, log processing, text preprocessing, and system-level file analytics.

**Learning Outcome**

Students will learn how to manually read files and count lines, words, and characters without STL while handling file-open errors correctly.

**Post Laboratory Work Description**

Submit counts of characters, words, and lines for at least two different test files and write a short reflection on the difference between STL vs manual processing.

## Practical 5.3: Manage an Inventory System

**Problem Definition**

Develop a file-based inventory management system that allows users to add items, view all items, and search for an item by its name. Each inventory record must store the item's **name**, **quantity**, and **price**, and these records will be stored sequentially inside a text file.

The program must support three operations:

1. **Add Item:** Take item details from the user and append the record to the inventory file.
2. **View All Items:** Read the inventory file sequentially and display every stored item.
3. **Search Item by Name:** Read each record, compare item names, and display the matching item's details.

**Key Questions / Analysis / Interpretation**

1. How do we read and write text files using ifstream and ofstream without STL containers?
2. What is the logic for manually comparing item names (case-sensitive or case-insensitive)?
3. How can we structure file records so they are easy to search and display?
4. What is the impact of appending vs rewriting the entire file?

**Supplementary Problems**

- Delete an item from the inventory file (requires rewriting the file).
- Modify the price or quantity of an existing item.
- Count total number of items in the inventory.
- Display all items with quantity below a threshold (low stock alert).

**Key Skills to be Addressed**

Manual file handling, string parsing, basic record management, and non-STL search logic.

**Applications**

Retail inventory systems, file-based data logging, and basic database simulations.

**Learning Outcome**

Students will learn how to manually store, retrieve, and search structured records in a file without STL by using fundamental file handling and string operations.

**Post Laboratory Work Description**

Add at least five items to the file, view all items, and successfully search for two different item names while submitting screenshots of each operation.

## Practical 5.4: Program to Format and Display Student Marks Using Stream Manipulators

**Problem Definition**

Write a C++ program to format and display a list of students with their marks using built-in stream manipulators such as endl, setw, setfill, and setprecision for neat tabular output. Additionally, design a **user-defined manipulator** named currency() that prepends a currency symbol (₹, $, etc.) before a numeric value. Demonstrate this manipulator while printing fees or any monetary field.

**Key Questions / Analysis / Interpretation**

1. How do built-in manipulators (setw, setfill, setprecision) affect table alignment and numeric formatting?
2. What considerations are required when formatting monetary values or decimal places?

**Supplementary Problems**

- Add alignment for columns using left and right manipulators.
- Add a column for percentage and format it to 2 decimal places.
- Format a complete fee receipt with borders created using setfill().

**Key Skills to be Addressed**

Formatted output control, stream manipulator usage, custom manipulator creation, table design, and precision handling.

**Applications**

Report card printing, fee receipts, formatted console dashboards, and financial report generation.

**Learning Outcome**

Students will learn how to use built-in and custom stream manipulators to create professionally formatted tables and monetary displays in C++.

**Post Laboratory Work Description**

Students must generate a formatted table for at least five students and demonstrate the usage of currency() for printing fees or other monetary values; screenshots of formatted outputs must be included.

# Practical 6: Inheritance

## Practical 6.1

A design engineering team needs a program that can efficiently compute and manage the areas of multiple circles used in various stages of product development. To accomplish this, they adopt an object-oriented approach that emphasizes modularity, reusability, and structured design.

The system must have two classes:

1. **A base class (Shape)**
    - Stores and manages the radius.
    - Ensures controlled access to the radius through encapsulation.
2. **A derived class (Circle)**
    - Inherits the radius from the base class.
    - Provides a method to calculate and return the area of a circle.

The engineers plan to handle **multiple circles**, exploring two approaches:

- A **dynamic method** using flexible runtime storage (e.g., dynamic arrays/vectors).
- A **static method** using fixed-size arrays.

The system should allow:

- Accepting the radius values of multiple circles.
- Computing their areas using the Circle class.
- Displaying the area for each circle clearly.

**Key Questions**

- How does the base class ensure that the radius is protected from direct modification?
- What role does inheritance play in extending Shape into Circle?
- How does the Circle class apply mathematical logic while maintaining OOP structure?
- What are the advantages and disadvantages of dynamic vs. static storage for multiple objects?

**Supplementary Problems**

- Extend the program to classify circles as **small**, **medium**, or **large** based on area thresholds.
- Add a feature to compute and store **circumference** in addition to area.

- Introduce an option to update the radius of an existing circle and recalculate its area.
- Store and display all circle data (radius + area + circumference) in tabular format.

**Key Skills to Be Addressed**

- Class design and abstraction
- Encapsulation and inheritance
- Dynamic and static data handling

**Applications**

- Graphics and rendering engines
- Engineering simulations
- Robotics and sensor modelling
- Educational software for geometry visualization

**Learning Outcome**

Students will be able to compare dynamic and static approaches for managing multiple objects and apply mathematical functions within object-oriented frameworks.

**Post Laboratory Work Description**

- Why encapsulation is essential for controlling radius input.
- How inheritance helped in organizing the solution.
- What difference they observed between dynamic and static circle management?
- What test cases they used (e.g., radius = 0, large radius values, multiple circles).

## Practical 6.2

**Problem Definition**

A rapidly expanding organization seeks a structured digital solution to represent its internal hierarchy. To model this progressively layered structure, the development team adopts a multilevel inheritance approach that mirrors real organizational roles.

The system is designed using three levels:

1. **Base Level – Person Class**
    - Stores fundamental details such as *name* and *age*.
    - Ensures these attributes are initialized properly through constructors.
2. **Intermediate Level – Employee Class**
    - Inherits from *Person*.
    - Introduces an *employee ID* to uniquely identify each employee.
    - Ensures seamless propagation of base-class attributes.
3. **Top Level – Manager Class**

- Inherits from *Employee*.
- Adds managerial information, particularly the *department overseen*.
- Provides methods to display complete hierarchical details.

The system must support:

- Constructor chaining across all levels to correctly initialize all data fields.
- Display functions at each level to outline the hierarchy and provide complete clarity on an individual's role.
- Two strategies to manage multiple managers:
  - An **efficient retrieval-based method** using employee identifiers.
  - A **static, straightforward storage method** for simpler use cases.

This experiment helps learners understand how multilevel inheritance can represent real-world hierarchical structures with clarity and modularity.

**Key Questions**

- How does multilevel inheritance ensure smooth propagation of data across Person → Employee → Manager?
- Why are constructors essential for initializing attributes at each level?
- How does the display function differ at each hierarchical step?
- What are the advantages and trade-offs of identifier-based retrieval versus static storage?

**Supplementary Problems**

- Extend the system to include **Assistant Managers**, adding another inheritance level.
- Add a feature to update the department of a manager dynamically.
- Introduce validation to ensure manager IDs remain unique across the system.
- Implement a search function to retrieve manager details based on department.

**Key Skills to Be Addressed**

- Multilevel inheritance and hierarchical modeling
- Constructor chaining and initialization flow
- Role-based information display and method overriding

**Applications**

- Corporate organizational mapping software
- Role-based access or workflow systems

**Learning Outcome**

Students will be able to model real-world hierarchical structures using multilevel inheritance.

**Post Laboratory Work Description**

- How constructors ensured accurate initialization at every inheritance level.
- The importance of maintaining clear hierarchy mapping in OOP models.
- The difference observed between retrieval-based and static manager storage.
- The test cases used to verify the correctness of the display operations.

## Practical 6.3

**Problem Definition**

A vehicle manufacturing company aims to build a structured software system for organizing and managing information about cars produced under its brand. To achieve this, a hierarchical class design was developed, capturing the essential components of a vehicle through Object-Oriented Programming.

The system is based on the following structure:

1. **Fuel Class**
    - Stores the type of fuel a vehicle uses (e.g., petrol, diesel, electric).
    - Ensures controlled initialization using constructors.
2. **Brand Class**
    - Maintains the brand or model name of the vehicle.
    - Provides secure handling of branding information.
3. **Car Class (Hybrid Inheritance)**
    - Derives from both *Fuel* and *Brand* classes.
    - Integrates fuel type and brand name into a unified entity.
    - Contains methods to display complete car details.

Constructors at each level ensure correct initialization and smooth integration of inherited attributes. To simulate a real-world scenario such as a service queue, multiple cars are processed sequentially. Two methods for managing the sequence—one using an efficient structured approach and the other using a direct static method—are compared to evaluate organizational efficiency in handling vehicle data.

**Key Questions**

- How does hybrid or multiple inheritance allow Car to integrate data from both Fuel and Brand?
- Why are constructors important for proper initialization across base and derived classes?
- How does the display function combine and present attributes from all inherited classes?
- How does a structured queue-based approach differ from a static storage method when handling multiple cars?

**Supplementary Problems**

- Add a *ServiceHistory* class and extend Car to include the last service date.
- Introduce validation to ensure that only valid fuel types are accepted.

- Implement a priority-based service queue (e.g., electric vehicles first).
- Add a feature to update or change the brand name for specific models.

**Key Skills to Be Addressed**

- Data organization and object integration
- Queue/structure-based data processing

**Applications**

- Vehicle management systems
- Automobile service center software
- Fleet management applications
- Manufacturing data modules
- Transport analytics platforms

**Learning Outcome**

Students will be able to implement hybrid inheritance to integrate multiple independent features.

**Post Laboratory Work Description**

Students must submit a reflection on:

- How constructors ensured correct initialization of all car attributes.
- The role of hybrid inheritance in combining independent classes.
- How the queue-based approach improved sequential processing.
- Test cases used to verify correct output and data handling.

## Practical 6.4

**Problem Definition**

To design a structured and user-friendly banking system, a hierarchical model was proposed to represent real-world account operations. At the core of the design is a **Base Account Class**, responsible for storing fundamental attributes such as *account number* and *current balance*, ensuring these details are properly encapsulated for security and controlled access.

Building upon this foundation, two specialized account types were introduced:

1. **Savings Account**
    - Extends the base class.
    - Adds an *interest rate* as an additional feature.
2. **Current Account**
    - Also inherits from the base class.
    - Includes an *overdraft limit*, supporting customers who require extended withdrawal flexibility.

Constructors and destructors were implemented in each class to manage initialization and clean-up of objects, ensuring that account resources are properly maintained throughout their lifecycle. Core banking operations—*deposit*, *withdraw*, and *display details*—were provided for both account types to simulate realistic financial transactions.

To enhance functionality, a mechanism to track and manage **transaction history** was integrated. This allowed users to review their recent operations and perform actions such as *undoing the last transaction*. By simulating transaction storage using an organized sequence, students explored different approaches to managing and structuring data across account types.

**Key Questions**

- How does the base class ensure secure encapsulation of account details?
- In what ways do the Savings and Current Account classes extend the functionality of the base class?
- How do constructors and destructors contribute to resource management in banking applications?
- How can transaction history be organized to support undo operations?
- What are the advantages of using structured data storage for bank transactions?

**Supplementary Problems**

- Add a feature to automatically apply interest to savings accounts at the end of the month.
- Implement overdraft penalty charges in current accounts.
- Allow users to export their last 10 transactions as a summary report.
- Introduce PIN-based authentication before performing any operation.
- Extend the system to support fixed deposit accounts using inheritance.

**Key Skills to Be Addressed**

- Constructor and destructor usage
- Polymorphism

**Applications**

- Banking management systems
- Digital financial platforms
- ATM and mobile banking software

**Learning Outcome**

Students will be able to implement inheritance to model different types of bank accounts.

**Post Laboratory Work Description**

Students are required to write a reflection covering:

- How encapsulation was used to safeguard account data.
- How inheritance differentiated savings and current account behaviors.

- The importance of constructors/destructors in financial system design.
- How transaction history was implemented and tested.
- Their test cases for deposit, withdrawal, overdraft, and undo operations.

# Practical 7: Polymorphism

### Practical 7.1: Problem Definition

A software development team aimed to create a flexible arithmetic utility demonstrating the concept of **function overloading**. To achieve this, a class-based calculator system was developed that supports multiple versions of the *add()* function. Each overloaded function accepts different input types—such as integers, floating-point numbers, or mixed combinations—ensuring the calculator adapts seamlessly to varying input scenarios.

The practical requires students to instantiate the calculator class, invoke the appropriate overloaded functions under multiple test cases, and store the computed results. To maintain clarity and organization, the system includes methods for sequentially processing and displaying the outcomes.

## Key Questions

- How does function overloading improve flexibility in arithmetic operations?
- What rules does C++ follow while selecting the appropriate overloaded function?
- How can mixed data types (int, float) be handled efficiently?
- How can results be stored and displayed in a structured way?

## Supplementary Problems

- Add overloaded subtract(), multiply(), and divide() methods.
- Introduce validation for division by zero.
- Implement a feature to compute the sum of an array using an overloaded function.

## Key Skills to Be Addressed

- Function overloading
- Class design
- Method invocation
- Handling mixed data types

## Applications

- Scientific calculators
- Financial computation engines

- Data processing applications

## Learning Outcome

Students will be able to use function overloading to design flexible utilities and handle different input types efficiently.

## Post Laboratory Work Description

- Students must reflect on how overloading improves usability and how they validated the correctness of each overloaded function.

## Practical 7.2: Problem Definition

A mathematical research group sought to build a computational model for managing **complex numbers** efficiently. To enable intuitive and user-friendly operations, a **Complex** class was created encapsulating real and imaginary parts.

The system includes key **operator overloads**, such as:

- + for adding two complex numbers
- – for subtracting complex numbers

These features provide a natural way to perform arithmetic on complex values, similar to built-in data types. Students must implement these operators, test them with multiple examples, and optionally explore managing collections of complex objects for batch operations.

## Key Questions

- How does operator overloading improve usability in numeric classes?
- What rules govern binary operator overloading?

## Supplementary Problems

- Overload *= and += operators.
- Create an array or vector of complex numbers for batch addition.
- Implement magnitude() and phase() functions.

## Key Skills to Be Addressed

- Operator overloading
  Object-oriented number modeling
  Input/output customization

## Applications

- Scientific simulations
  Signal processing systems
  Electrical engineering computations

## Learning Outcome

Students will understand how operator overloading brings built-in functionality to user-defined types.

## Post Laboratory Work Description

- Students reflect on usability improvements and edge-case testing for complex arithmetic.

## Practical 7.3: Problem Definition

A weather application required a system for smooth **temperature conversion** between Celsius and Fahrenheit. To support bi-directional conversion, two classes were developed:

- **Celsius**
- **Fahrenheit**

Each class uses **type-conversion operator overloading** to convert one unit to the other seamlessly. Additionally, the equality operator == is overloaded to compare temperatures.

To manage multiple converted objects, two storage approaches were explored:

1. A **dynamic queue** for FIFO-based processing
2. A **fixed array** for static storage

Students must implement conversions, comparisons, and explore storing collections of converted temperature values.

## Key Questions

- How does operator overloading support custom type conversion?
- What is the significance of converting both ways (Celsius ↔ Fahrenheit)?
- Why use a queue for managing temperature conversions?
- How are temperature comparisons managed correctly?

## Supplementary Problems

- Add Kelvin class and implement conversions.
- Store conversion history in a file.
- Add > and < operators for comparison.

## Key Skills to Be Addressed

- Type conversion
  Operator overloading
  Dynamic vs static storage

## Applications

- Weather forecasting systems
  IoT sensor data processing
  Environmental monitoring tools

## Learning Outcome

- Students will learn how to overload type-conversion operators and manage collections of converted objects.

## Practical 7.4: Problem Definition

A team of developers was assigned to design a flexible area-calculation system capable of handling multiple **geometric shapes**. To ensure extensibility, a **base class Shape** was created containing a **virtual Area () function**. Derived classes include:

- **Rectangle** – area computed from length × width
- **Circle** – area computed from $\pi \times \text{radius}^2$

Polymorphism allows using a single **Shape*** reference to compute types of areas uniformly. This design supports future shape extensions without changing existing logic.

The system also explores two collection-management strategies:

1. **Dynamic storage** allowing flexible resizing
2. **Static storage** using fixed arrays

Students implement these classes, test polymorphic area computation, and compare collection handling strategies.

## Key Questions

- How does polymorphism unify shape handling?
- Why is Area () declared virtual?
- What benefits arise from using a Shape pointer?
- How do dynamic and static collections differ in flexibility?

## Supplementary Problems

- Add Triangle, Square, or Ellipse classes.
- Implement perimeter () using virtual functions.
- Manage shape deletion using smart pointers.

## Key Skills to Be Addressed

- Polymorphism
  Virtual functions
  Inheritance
  Memory management
  Dynamic vs static containers

## Applications

- Graphics libraries
  Geometry modeling tools
  Simulation systems

## Learning Outcome

Students will understand polymorphism and design extensible shape systems.

## Practical 7.5: Problem Definition

To support the diverse assessment needs across academic programs, an advanced grading system was designed using an object-oriented approach. At the foundation of this design is an **abstract base class**, representing a generic grading framework. This class contains a protected member for storing marks and declares a **pure virtual function** for computing grades, ensuring that all derived classes must define their own grading logic.

Two specialized classes were implemented to handle distinct evaluation criteria:

1. **Undergraduate Class**
   - Inherits from the abstract grading class.
   - Implements its own grade computation based on undergraduate academic standards.
2. **Postgraduate Class**
   - Also inherits from the abstract base class.
   - Defines a more rigorous grading methodology suitable for postgraduate students.

Users can input student information, compute grades according to their academic level, and maintain a collection of student records. This setup demonstrates the power of abstraction and runtime polymorphism in modeling flexible academic evaluation systems.

### Key Questions

- How does the abstract class enforce a uniform structure for grade computation?
- Why is a pure virtual function essential in this design?
- How do undergraduate and postgraduate classes differ in their grading logic?
- How does polymorphism simplify record management for students of various academic levels?

- What are the benefits of using protected members in an abstract class?

**Supplementary Problems**

- Add a **research scholar** class with its own grading pattern.
- Allow each student record to store additional details such as course name or credit weightage.
- Implement weighted grade computation using credits.
- Extend the system to calculate class averages for each academic level.
- Provide an option to display a summary of all stored student records.

**Key Skills to Be Addressed**

- Abstraction and pure virtual functions
- Polymorphism and dynamic binding
- Class hierarchy and method overriding
- Secure handling of academic data
- Organizing and managing collections of derived-class objects

**Applications**

- University result processing systems
- Academic evaluation tools
- Online grading and assessment platforms
- Institutional student performance analytics
- Course management systems

**Learning Outcome**

Students will be able to implement abstract classes and pure virtual functions.

**Post Laboratory Work Description**

Students must prepare a reflection describing:

- The role of abstraction in enforcing grading structure.
- Differences in grading logic among UG and PG classes.
- How polymorphism simplified computation and record handling.
- Sample test cases used for each student category.

## Practical: 7.6 Base Class with Virtual Destructor & Resource Management

**Problem Definition**

A system requires safe deletion of objects through base-class pointers in a polymorphic setup. You must design a base class **Base** and a derived class **Derived**, where Derived dynamically allocates a resource using **raw pointers** (new).

The practical must first demonstrate that **if the destructor in Base is NOT virtual**, then deleting a Base* that actually points to a Derived object results in **partial destruction** — the Derived destructor does not execute, causing **resource leakage**.

Then, modify the base class to include a **virtual destructor**, showing that deleting the same Base* now correctly invokes the Derived destructor and releases all resources safely.

This practical helps students understand **why virtual destructors are essential** in polymorphic base classes and how manual resource management works using raw pointers.

**Key Questions / Analysis / Interpretation**

1. Why is a virtual destructor required when deleting objects through a base-class pointer?
2. What happens if the base destructor is non-virtual?
3. How does Derived manage dynamic memory, and why must its destructor be called?
4. How do constructor/destructor call orders affect resource cleanup?

**Supplementary Problems**

- Add multiple resources in Derived and verify cleanup correctness.
- Introduce a second derived class and test polymorphic cleanup.
- Implement a simple static counter to track resource leaks.

**Key Skills to be Addressed**

- Understanding polymorphism and destructor chaining
- Manual memory management using raw pointers (new, delete)
- Correct design of base classes for inheritance
- Debugging destructor behavior and detecting leaks

**Applications**

- Memory-safe class hierarchy design
- System-level C++ programming
- Foundations for RAII and smart-pointer design
- Legacy C++ maintenance where raw pointers are used

**Learning Outcome**

Students will be able to demonstrate why a base class destructor should be virtual in polymorphic use and identify resource leaks caused by improperly designed destructors.

**Post Laboratory Work Description**

- Program output showing behavior before and after making the destructor virtual.
- A short explanation of why resource leakage occurs.
- A screenshot or printout of constructor/destructor call sequence.

# Practical 8: Exception Handling

## Practical 8.1: Exception Handling for Division with Direct User Input

### Problem Definition

Write a C++ program that takes **two integers directly from user input** and performs division while handling runtime exceptions. The program must detect and handle two error situations:

1. **Division by zero** when the user enters 0 as the denominator.
2. **Invalid input** when the user enters non-integer values.

The division must be performed **immediately during input extraction**, meaning the program should not store the values first for later processing—exceptions must be checked and handled at the moment of extraction or evaluation.

### Key Questions / Analysis / Interpretation

1. How can exceptions be used to detect invalid input via cin.fail()?
2. Why must division by zero be handled before performing the division?
3. What kind of exceptions are appropriate for mathematical and input-related errors?
4. How can the program avoid undefined behavior when invalid data is entered?

### Supplementary Problems

- Extend the program to handle floating-point input as well.
- Wrap the entire division logic in a separate function and throw custom exceptions.
- Retry input until valid values are entered.
- Provide meaningful messages to guide the user when errors occur.

### Key Skills to be Addressed

Exception handling, input validation, runtime error detection, direct division execution, and safe program flow using try–catch.

### Applications

Secure calculators, form data validation, financial and scientific tools that must avoid division errors.

**Learning Outcome**

Students will learn how to detect and handle division-by-zero and invalid input errors using exceptions while performing direct real-time division operations.

**Post Laboratory Work Description**

Students must test the program with at least five different input combinations—including invalid input and zero denominator—and submit screenshots showing the exception messages.

## Practical: 8.2 Square Root Calculation with Custom Exception Handling

**Problem Definition**

Design a C++ program that calculates the square root of a user-entered number and throws a custom exception whenever the input is negative. The program must define a custom exception class named **NegativeNumberException** and handle it using a try–catch block. Each input must be processed **independently**, meaning the user enters one number at a time and the program immediately checks and handles negative values before attempting to compute the square root.

**Key Questions / Analysis / Interpretation**

1. Why can negative numbers not be processed by the standard square root function in real-number computation?
2. How does a custom exception differ from built-in exceptions in C++?
3. How should the exception class be designed to store and display error information?
4. How can individual inputs be validated before performing mathematical operations?

**Supplementary Problems**

- Modify the program to allow retrying until the user provides a valid number.
- Add a separate exception for extremely large numbers.
- Display a detailed error message showing the invalid value.
- Extend support to compute cube roots and handle negatives appropriately.

**Key Skills to be Addressed**

Custom exception creation, try–catch handling, input validation, mathematical operation safety, and independent processing of each input.

**Applications**

Scientific computation tools, user input validation systems, calculators, and software requiring safe mathematical execution.

**Learning Outcome**

Students will learn to design and use custom exceptions to safely handle invalid numerical inputs while executing mathematical operations such as square root calculation.

**Post Laboratory Work Description**

Students must test at least six inputs, including negative numbers, and submit screenshots showing successful handling of custom exceptions and valid square root outputs.

## Practical 8.3: Simulate a Banking System with Exception Handling and Stack Unwinding

**Problem Definition**

Design and implement a C++ program that simulates a simple banking system using a BankAccount class to manage an account balance. The class must support deposit and withdraw operations, and should throw exceptions for invalid operations such as withdrawing more than the available balance or depositing invalid amounts if required. The program must also demonstrate **stack unwinding** by logging each function call involved in the transaction process when an exception is thrown. A basic array must be used to store transaction logs (such as function entry/exit messages and transaction results), which are displayed at the end of execution to show how control flowed during exception propagation.

**Key Questions / Analysis / Interpretation**

1. When should an exception be thrown in banking operations like withdrawal or deposit?
2. How does stack unwinding work when an exception propagates through multiple function calls?
3. How can function entry and exit be logged to observe the unwinding process?
4. How can a simple array be used to store and later display transaction and call-trace logs?

**Supplementary Problems**

- Create a custom exception class (for example, InsufficientFundsException) with an appropriate message and attempted amount.
- Implement a transfer function that moves money between two BankAccount objects and demonstrate exception handling if the source has insufficient funds.
- Add a limit for maximum daily withdrawal and throw an exception if it is exceeded.
- Store transaction types (deposit/withdraw) and amounts in parallel arrays and display a mini-statement at the end.

**Key Skills to be Addressed**

Exception throwing and catching, class design for financial operations, understanding and demonstrating stack unwinding, and using basic arrays to record and display logs and transaction history.

**Applications**

Banking and financial system simulations, secure transaction handling, error tracing in layered architectures, and teaching exception safety in real-world scenarios.

**Learning Outcome**

Students will learn to design a simple banking system with robust exception handling, observe stack unwinding through logged function calls, and record transaction logs using basic arrays.

**Post Laboratory Work Description**

Students must implement at least one successful deposit, one successful withdrawal, and one failed withdrawal (due to insufficient balance). They should capture and submit the transaction logs showing function call entries/exits and exception handling, along with final account balance and printed logs.

## Practical 8.4: Read and Process Data from a File with Exception Handling

**Problem Definition**

Write a C++ program that reads data from a text file and processes each line as it is read. The program must validate input while reading and handle these error conditions:

- **File not found** (or cannot be opened) — prompt the user to re-enter the file path (retry mechanism).
- **Corrupted data** — e.g., non-numeric values where numbers are expected; handle such lines gracefully (skip, report, or ask for correction).

The program should process the file **directly line-by-line** (no storing entire file in memory). For each line, validate fields, perform the required numeric operations (e.g., summation, averaging, or any simple numeric processing defined by the instructor), and record/report any invalid lines. Implement a user-friendly retry mechanism so that if the file cannot be opened, the user may enter another path (or exit).

**Key Questions / Analysis / Interpretation**

1. How to detect and handle failure to open a file (ifstream::open / is_open) and implement a retry loop?
2. What constitutes corrupted data in the context of the required processing, and how can it be detected while parsing a line?
3. How to validate numeric fields read from a string (e.g., using std::stringstream extraction checks or manual digit checks)?

4. What is the desired behavior on encountering a corrupted line: skip, log and continue, or stop and ask user action?
5. How to report errors clearly to the user (line number, content, reason)?

**Supplementary Problems**

- Keep a fixed-size error log (array) to save details of corrupted lines (line number and reason) and print at the end.
- Allow the user to choose the processing action on corrupted lines: skip / replace with default / abort.
- Add a mode to reprocess only the previously failing lines after the file is corrected.
- Implement simple aggregation (sum, count, average) of numeric fields while ignoring bad lines and report final stats.
- Track and display progress (number of lines processed vs total) if the file size is known.

**Key Skills to be Addressed**

Robust file I/O, line-by-line parsing and validation, exception handling and user-interactive retry logic, error reporting, and low-memory streaming processing.

**Applications**

Log file processing, CSV data ingestion, ETL (extract-transform-load) pre-processing, and resilient batch data pipelines.

**Learning Outcome**

Students will learn to read and validate data line-by-line from a file with robust handling for file-open errors and corrupted lines, implementing a retry mechanism and clear error reporting.

**Post Laboratory Work Description**

Students must submit: program source, sample input file(s) (one correct, one containing some corrupted lines), program output showing retry behavior when a wrong path is given, and an error-log output summarizing corrupted lines.

# Practical 9: Standard Template Library

## Practical 9.1: Reverse a std::vector of Integers Using Iterators

### Problem Definition

Develop a C++ program that reads a sequence of integers from the user, stores them inside a std::vector<int>, and reverses the vector using iterator-based operations. The program must demonstrate **two reversal methods**:

1. **Using std::reverse()** from the library.
2. **Manual reversal using iterators**, swapping elements from beginning to end.

### Key Questions / Analysis / Interpretation

1. How do iterators provide pointer-like access to elements in a vector?
2. What is the difference between using std::reverse() and manually reversing with iterators?
3. How does iterator increment/decrement work during reversal?
4. Why are iterators preferred over raw pointers in STL containers?

### Supplementary Problems

- Reverse only a portion of the vector using iterator ranges.
- Check if the vector is a palindrome using iterators.
- Implement a custom reverse function that accepts two iterators as parameters.
- Display vector elements using different iterator types (normal, const, reverse).

### Key Skills to be Addressed

Iterator traversal, vector manipulation, STL algorithm usage, manual reverse logic, iterator safety, and understanding forward and reverse iteration.

### Applications

Data reordering, buffer reversal, algorithm preprocessing, stack simulation, and basic competitive programming techniques.

### Learning Outcome

Students will learn to manipulate vectors using iterators and apply both STL algorithms and manual iterator logic to reverse container elements.

**Post Laboratory Work Description**

Students must submit outputs for both reversal methods, provide their manual iterator-based implementation, and include a short note comparing the simplicity of std::reverse() with the logic required for manual reversal.

## Practical 9.2 Count the Frequency of Each Unique Word Using std::map

**Problem Definition**

Write a C++ program that reads a sentence from the user and counts the frequency of every unique word using std::map. Each word should be extracted from the input sentence, converted as needed (case handling optional unless specified), and stored as a **key** in the map with its **frequency** as the value. After processing the entire sentence, iterate through the map to display each word alongside its count.

**Key Questions / Analysis / Interpretation**

1. How does std::map automatically maintain key-value pairs and ensure sorted keys?
2. How can text be parsed into individual words for insertion into the map?
3. What is the role of map[word]++ in frequency counting?
4. How is iterator-based traversal used to display map contents?

**Supplementary Problems**

- Convert all words to lowercase before counting.
- Remove punctuation symbols from the sentence.
- Display the most frequent and least frequent word.
- Replace std::map with std::unordered_map and compare performance.

**Key Skills to be Addressed**

STL associative container usage, map insertion and frequency counting, iterator traversal, and basic text tokenization.

**Applications**

Text analytics, indexing, keyword extraction, counting occurrences in logs, and basic natural language processing.

**Learning Outcome**

Students will learn how to use std::map to store unique words and count their frequencies by inserting and iterating through key–value pairs.

**Post Laboratory Work Description**

Students must test the program with at least three sentences, show the resulting word-frequency output, and submit a short explanation of how std::map organizes and stores data.

## Practical 9.3: Remove Duplicate Elements from a List Using std::set

**Problem Definition**

Write a C++ program that reads a list of integers from the user, stores the values inside a std::set to automatically eliminate duplicates, and then either converts the set back into a std::vector or directly displays the elements using iterators. Since std::set maintains unique elements in sorted order, the program should clearly demonstrate how duplicates are removed and how iterator-based traversal displays the resulting unique list.

**Key Questions / Analysis / Interpretation**

1. How does std::set ensure uniqueness and automatic sorting of inserted elements?
2. What is the difference between inserting into a vector vs. inserting into a set?
3. How do iterators help in accessing elements inside a set?
4. When and why might one convert a set back into a vector?

**Supplementary Problems**

- Convert the std::set into a vector and sort it in descending order.
- Count how many duplicates were removed.
- Replace std::set with std::unordered_set and compare behavior.
- Insert random integers and measure time differences between vector and set insertions.

**Key Skills to be Addressed**

Use of STL associative containers, understanding uniqueness enforcement, iterator traversal, and converting between STL containers.

**Applications**

Duplicate removal in datasets, cleaning user input, preprocessing for algorithms requiring unique values, and ensuring distinct keys in data structures.

**Learning Outcome**

Students will learn to eliminate duplicates efficiently using std::set and access or convert the resulting unique elements through iterators or vectors.

**Post Laboratory Work Description**

Students must provide input containing duplicates, display the unique values using iterators, and optionally convert and print the data using a vector.

## Practical 9.4: Implement a Simple Directory Management System Using std::map and std::vector

**Problem Definition**

Create a C++ program that simulates a basic directory management system using std::map<std::string, std::vector<std::string>>, where each folder name acts as a **key** and the list of files in that folder acts as the **value**.
The program must provide interactive options to:

1. **Add a folder** to the directory structure.
2. **Add files** to an existing folder by inserting entries into the corresponding vector.
3. **Display all folders and their files**, showing the full directory contents.

The program must also demonstrate how folder names can be **sorted alphabetically** using the natural key ordering of std::map and iterated using map iterators.

**Key Questions / Analysis / Interpretation**

1. How does std::map store folder names in sorted order automatically?
2. Why is std::vector an appropriate structure to store multiple filenames inside each folder?
3. How do iterators help traverse folder–file relationships within a map?
4. How should the program behave if a folder already exists or if a file is added repeatedly?

**Supplementary Problems**

- Prevent duplicate file names within the same folder.
- Remove a file from a folder.
- Remove an entire folder from the directory.
- List only folder names without files.
- Display the total number of files in each folder.

**Key Skills to be Addressed**

Map–vector combined usage, iterator traversal, structured data organization, alphabetical ordering through associative containers, and menu-driven program design.

**Applications**

File system simulations, directory indexing, cloud storage metadata handling, menu-driven mapping systems, and hierarchical data representation.

**Learning Outcome**

Students will learn to build a simplified directory management system using std::map and std::vector, utilizing iterators to organize and display folder–file structures in alphabetical order.

**Post Laboratory Work Description**

Students must demonstrate folder creation, adding multiple files to at least two folders, and printing the entire directory structure using iterator traversal; screenshots of program output must be submitted.